SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED
COURSE CODE: MLA01

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)
Assessment Tool Weightage: 40%

Total Marks: 100 Time: 90 Mins No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:
*I **Kamalli shree V(192525030)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* ***Signature of the student:***

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis 8. Unification, Backtracking and Inference Accuracy	10% 10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	**Medical Diagnosis Expert System:** Develop a rule-based expert system that identifies possible diseases based onsymptoms, patient observations, and basic clinical information. Represent domainknowledge using production rules and implement both forward and backwardchaining to derive conclusions.
2	**Crop Disease Advisory System:** Develop an expert system that identifies possible crop diseases based on leafsymptoms, soil conditions, weather conditions, and visible plant characteristics.Construct suitable production rules and demonstrate diagnosis using forward andbackward chaining.
3	**Automobile Fault Diagnosis System:** Design an expert system that identifies probable vehicle faults based on symptomssuch as engine noise, starting problems, warning indicators, abnormal vibration,and reduced mileage. Implement the knowledge base using production rules andcompare forward and backward reasoning.
4	**Student Academic Advisory System:** Develop an expert system that recommends suitable academic interventions basedon attendance, internal marks, assignment performance, learning difficulties, andprevious academic performance. Use production rules and demonstrate bothforward and backward chaining.
5	**Cybersecurity Threat Diagnosis System:** Construct a rule-based expert system that identifies possible cybersecurity threatsfrom events such as repeated login failures, unusual network traffic, suspicious fileactivity, and unauthorized access attempts. Implement production rules anddemonstrate how forward and backward chaining arrive at a threat classification.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.

4. Prolog Implementation	15%	13–15% – Implements a fullyfunctional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementati on with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.

7. Procedural and Non-Procedu ral Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-pr ocedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstrati on.	0–4% – Paradigms are misunderstood ornot addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstratio n	5%	5% – Provides comprehensi ve test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstrati on with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadeq uate.
10. Documentati on, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentat io n and repository with minor issues.	2–3% – Basic documentation/ re pository with several omissions.	0–1% – Poor documentati on, missing repository, or inadequate presentation.
Total	100%				

Medical Diagnosis Expert System using Prolog

1. Domain Analysis

1.1 Problem Definition

The proposed system is a rule-based Medical Diagnosis Expert System developed using Prolog. The system identifies possible medical conditions based on symptoms provided by a patient. The medical knowledge is represented using facts and IF–THEN production rules.

The system uses an inference engine to reason over the available symptoms and derive possible conclusions using forward chaining and backward chaining.

The system is intended as an academic demonstration of a rule-based expert system and is not intended to replace professional medical diagnosis.

1.2 Entities

The major entities in the system are:

- Patient

- Symptoms
- Diseases/conditions
- Production rules
- Knowledge base
- Inference engine
- Diagnosis

1.3 Input Facts

The system considers symptoms such as:

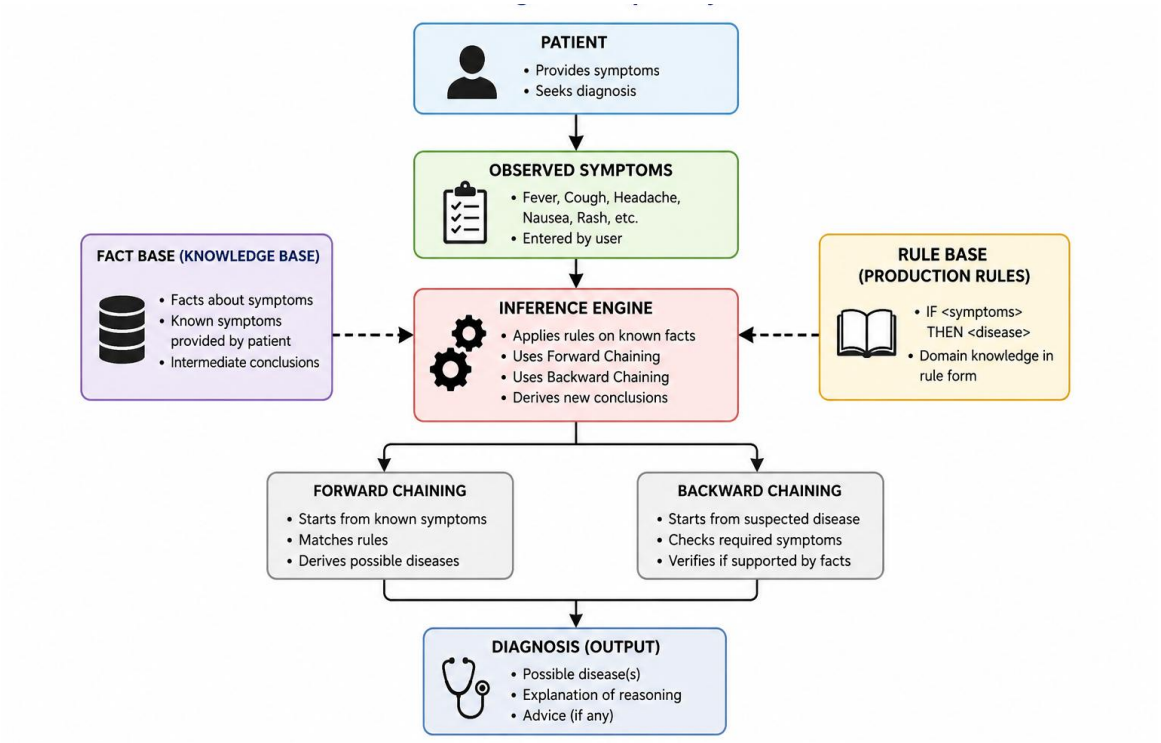
- fever
- cough
- sore_throat
- runny_nose
- body_pain
- headache
- nausea
- vomiting
- diarrhea
- rash

1.4 Possible Conclusions

The system contains rules for:

- Flu
- Common Cold
- Migraine
- Food Poisoning
- Viral Infection
- Measles-like Condition

1.5 Domain Model



2. Knowledge Acquisition

2.1 Knowledge Specification

The system uses simplified symptom-condition relationships to demonstrate expert-system reasoning.

Condition	Symptoms Used
Flu	Fever, cough, body pain
Common Cold	Cough, runny nose, sore throat
Migraine	Headache, nausea, vomiting
Food Poisoning	Nausea, vomiting, diarrhea
Viral Infection	Fever, cough, sore throat, body pain
Measles-like Condition	Fever, rash, cough

These relationships are converted into production rules in the next stage.

Note: The symptom combinations are simplified for an academic expert-system demonstration and should not be treated as a real diagnostic system.

3. Knowledge Representation

3.1 Facts

The available symptoms are represented as Prolog facts:

- symptom(fever).
- symptom(cough).
- symptom(sore_throat).
- symptom(runny_nose).
- symptom(body_pain).
- symptom(headache).
- symptom(nausea).
- symptom(vomiting).
- symptom(diarrhea).
- symptom(rash).

3.2 Production Rules

- rule(flu, [fever, cough, body_pain]).
- rule(common_cold, [cough, runny_nose, sore_throat]).
- rule(migraine, [headache, nausea, vomiting]).
- rule(food_poisoning, [nausea, vomiting, diarrhea]).

- rule(viral_infection, [fever, cough, sore_throat, body_pain]).
- rule(measles_like_condition, [fever, rash, cough]).

Example:

IF fever AND cough AND body_pain

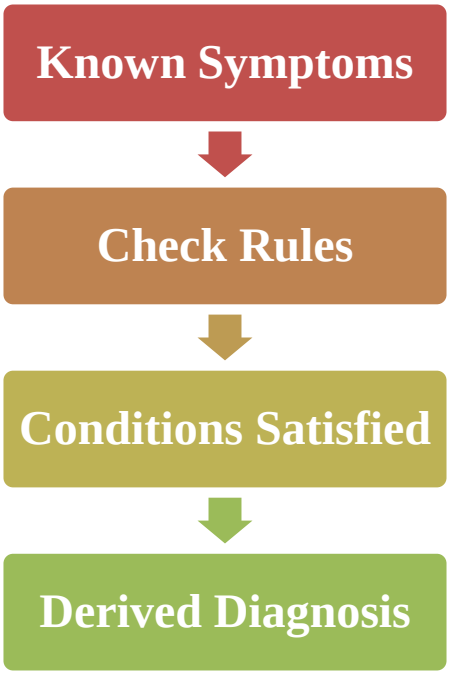
THEN flu

```
==============================================
        MEDICAL DIAGNOSIS EXPERT SYSTEM
==============================================
  Rule-Based Expert System using Prolog
----------------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------------
  Enter option: 1.

========================================
        PATIENT SYMPTOM INPUT
========================================
Available symptoms:
fever, cough, sore_throat, runny_nose,
body_pain, headache, nausea, vomiting,
diarrhea, rash

Enter number of symptoms: |: 3.
Enter symptom: |: fever.
Enter symptom: |: cough.
Enter symptom: |: body_pain.

  Patient Symptoms:
  -----------------
  -> fever
  -> cough
  -> body_pain

  Possible Diagnosis:
  -------------------
  -> flu
```

4. Inference Design

4.1 Forward Chaining

Forward chaining starts with known patient symptoms and applies production rules whose conditions are satisfied to derive new conclusions.



For example:

Fever + Cough + Body Pain

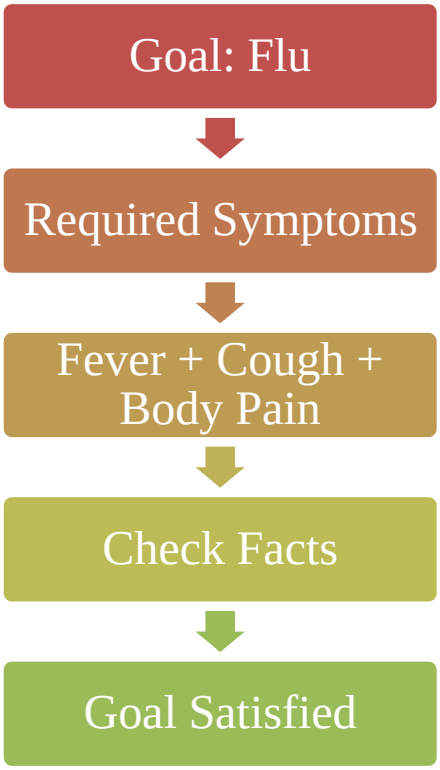
↓

Flu

```
----------------------------------------
  Rule-Based Expert System using Prolog
----------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------
  Enter option: |: 2.

========================================
        FORWARD CHAINING
========================================
  Starting from known patient symptoms...
  [Derived] flu

  Forward chaining completed.
```

4.2 Backward Chaining

Backward chaining starts with a proposed diagnosis and works backward to determine whether the symptoms required by the corresponding rule are present




```
==============================================
        MEDICAL DIAGNOSIS EXPERT SYSTEM
==============================================
  Rule-Based Expert System using Prolog
----------------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------------
  Enter option: |: 3.
Enter diagnosis to verify: |: flu.

==========================================
        BACKWARD CHAINING
==========================================
  Goal: flu

  Explanation for flu:
  Required symptoms:
  -> fever
  -> cough
  -> body_pain

  Result: Goal satisfied.
  Diagnosis flu is supported by the known symptoms.
```

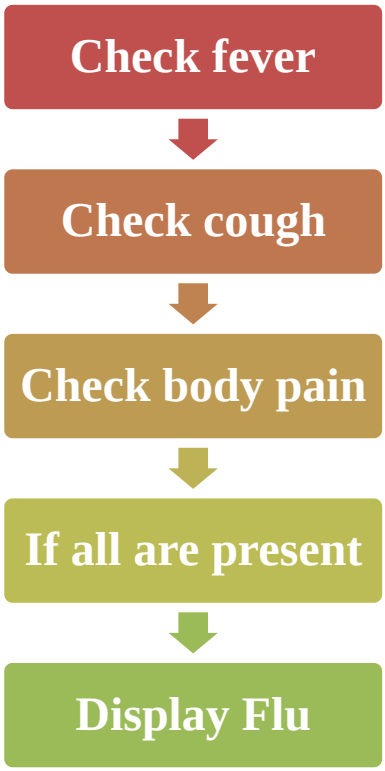
5. Paradigm Analysis

5.1 Declarative Approach

Prolog follows a declarative approach. The programmer specifies facts and relationships, while the inference mechanism determines how conclusions are derived.

Example:

rule(flu, [fever, cough, body_pain]).



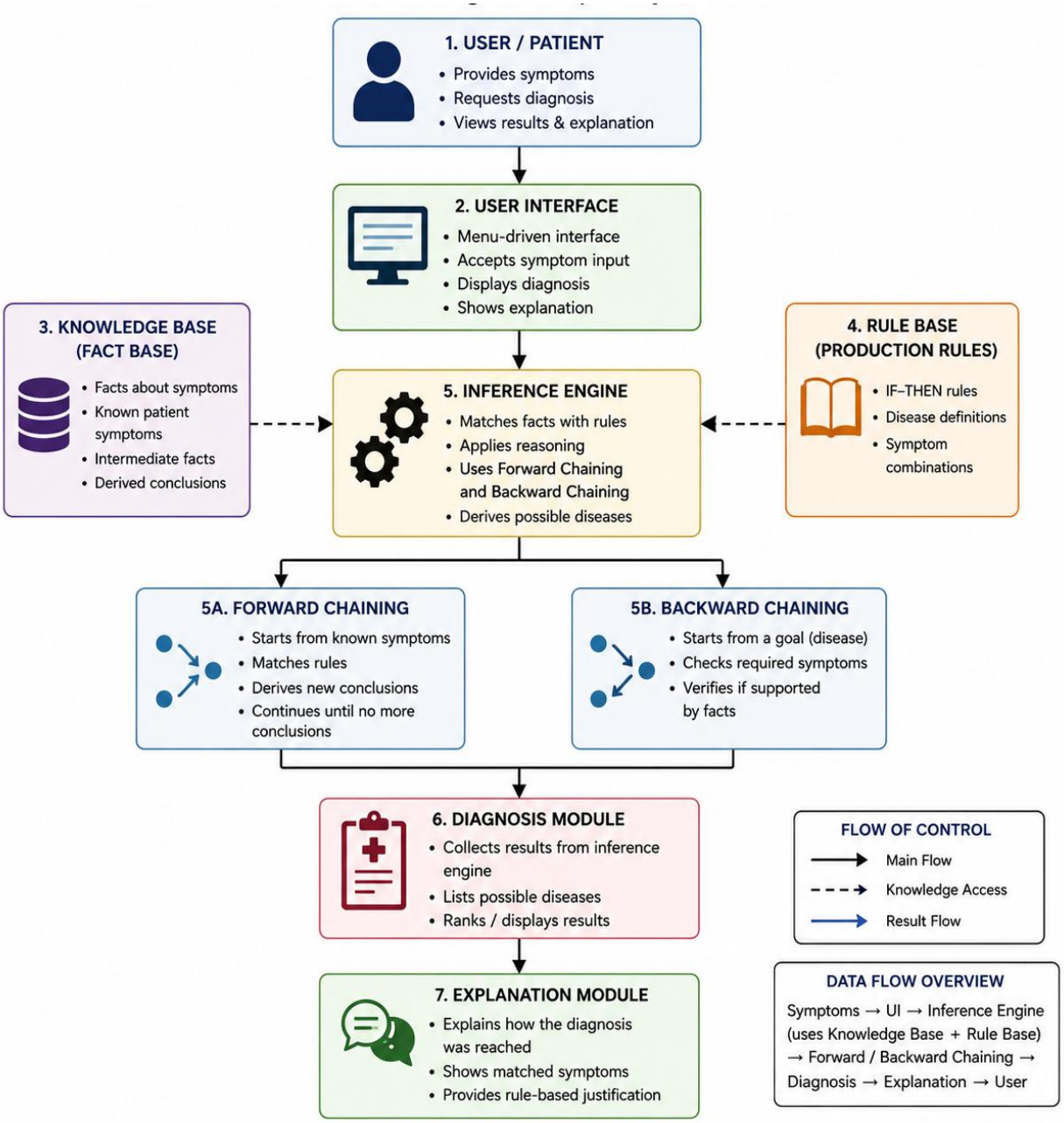
5.3 Comparison

Feature	Prolog	Procedural Language
Programming style	Declarative	Procedural
Focus	What is true	How to perform the task
Knowledge	Facts and rules	Conditions and procedures
Inference	Rule-based	Explicit control flow
Expert-system representation	Natural	Requires additional logic

.

6. System Development

6.1 System Architecture



6.2 Start the System

```
101 ?- consult(['c:/Users/Admin/Downloads/AT/medical_expert.pl']).
true.

102 ?- start.

==============================================
       MEDICAL DIAGNOSIS EXPERT SYSTEM
==============================================
  Rule-Based Expert System using Prolog
----------------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------------
  Enter option: 1.
```

6.3 Enter a Patient Case

```
==============================================
       MEDICAL DIAGNOSIS EXPERT SYSTEM
==============================================
  Rule-Based Expert System using Prolog
----------------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------------
  Enter option: 1.

==========================================
       PATIENT SYMPTOM INPUT
==========================================
Available symptoms:
fever, cough, sore_throat, runny_nose,
body_pain, headache, nausea, vomiting,
diarrhea, rash

Enter number of symptoms: |: 3.
Enter symptom: |: fever.
Enter symptom: |: cough.
Enter symptom: |: body_pain.

  Patient Symptoms:
  -----------------
  -> fever
  -> cough
  -> body_pain

  Possible Diagnosis:
  -------------------
  -> flu
```

7. Testing

Test Case	Symptoms	Expected Diagnosis
TC1	Fever, Cough, Body Pain	Flu
TC2	Cough, Runny Nose, Sore Throat	Common Cold
TC3	Headache, Nausea, Vomiting	Migraine
TC4	Nausea, Vomiting, Diarrhea	Food Poisoning

```
==========================================
         SYSTEM TESTING
==========================================

Test Case 1: Flu

  Patient Symptoms:
  ------------------
  -> fever
  -> cough
  -> body_pain

  Possible Diagnosis:
  ---------------------
  -> flu

Test Case 2: Common Cold

  Patient Symptoms:
  ------------------
  -> cough
  -> runny_nose
  -> sore_throat

  Possible Diagnosis:
  ---------------------
  -> common_cold

Test Case 3: Migraine

  Patient Symptoms:
  ------------------
  -> headache
  -> nausea
  -> vomiting

  Possible Diagnosis:
  ---------------------
  -> migraine
```

```
Test Case 4: Food Poisoning

  Patient Symptoms:
  ------------------
  -> nausea
  -> vomiting
  -> diarrhea

  Possible Diagnosis:
  --------------------
  -> food_poisoning

==========================================
        ALL TESTS COMPLETED
==========================================
```

8. Evaluation

8.1 Correctness

The system successfully derives the expected condition when the symptoms required by a production rule are present.

8.2 Coverage

The knowledge base contains rules for six conditions:

- Flu
- Common Cold
- Migraine
- Food Poisoning
- Viral Infection
- Measles-like Condition

8.3 Consistency

The same set of symptoms produces the same rule-based conclusion because the same production rules are applied to the available facts.

8.4 Reasoning

The system supports:

Forward Chaining:
Symptoms → Rules → Diagnosis

and:

Backward Chaining:
Diagnosis → Required Symptoms → Verification

8.5 Evaluation Result

The test cases demonstrate that the expert system can apply its production rules to different symptom combinations and derive the corresponding conclusions. The reasoning is transparent because each conclusion is associated with a specific rule and its required symptoms.

The system is limited by its simplified knowledge base and therefore serves as an academic demonstration rather than a real medical diagnostic system.

Conclusion

The Medical Diagnosis Expert System was successfully developed using Prolog as a rule-based expert system. The system represents medical knowledge using facts and IF–THEN production rules and uses an inference engine to derive possible diagnoses from patient symptoms.

Both forward chaining and backward chaining were implemented to demonstrate different reasoning approaches. The system was tested with multiple symptom combinations, including flu, common cold, migraine, and food poisoning, and produced the expected rule-based conclusions.

The project demonstrates how knowledge representation, inference, and explanation facilities can be integrated into an expert system. Although the knowledge base is simplified for academic purposes, the system provides a clear example of how Prolog can be used to model and reason about domain-specific knowledge.

```
==============================================
       MEDICAL DIAGNOSIS EXPERT SYSTEM
==============================================
  Rule-Based Expert System using Prolog
----------------------------------------------
  1. Enter patient symptoms
  2. Forward chaining
  3. Backward chaining
  4. Show diagnosis
  5. Run test cases
  6. Clear patient data
  7. Exit
----------------------------------------------
  Enter option: |: 7.

Thank you for using the Medical Diagnosis Expert System.
==============================================
true .
```